

PeoplePay360 — Complete Project Guide

A single reference covering the stack, the database, every folder, the core functionality and the technical decisions behind them. Written to be read end to end before an evaluation or viva.

The other documents go deeper on individual areas — [architecture](#), [data-model](#), [api](#), [frontend](#), [operations](#). This one is the overview that ties them together.

1. What the project is

PeoplePay360 is an integrated **HR and Payroll operations platform**. One system handles the whole employee lifecycle and turns it into money paid:

Hire an employee → give them a contract and a working schedule → record their attendance and approved leave → run payroll for a period → produce validated payslips → email them as PDF.

The **employee record is the hub**. Everything else hangs off it, and payroll reads through it. The central claim of the project is that payroll figures are **derived from real recorded facts** (contract wage, actual attendance, approved leave) rather than typed in by hand.

The one-line pitch

A role-scoped HR and payroll system where the payroll engine is data-driven — salary rules are configurable rows in a database evaluated in sequence, not hardcoded formulas — with a full audit trail of every change.

2. Tech stack

Backend

Technology	Version	Why it is used
NestJS	11	Backend framework. Modular architecture, dependency injection, decorator-based routing, built-in guards and pipes.
TypeScript	5.7	Static typing across all three packages.
Prisma	6	ORM / database toolkit. Type-safe queries, schema-driven migrations.
PostgreSQL	16	Relational database.

Technology	Version	Why it is used
Passport + passport-jwt	—	JWT authentication strategy.
bcryptjs	—	Password hashing (10 salt rounds).
class-validator / class-transformer	—	Request body (DTO) validation.
Helmet	8	Security HTTP headers.
PDFKit	—	Generates payslip PDFs server-side.
@nestjs/swagger	11	Auto-generated OpenAPI docs at <code>/api/docs</code> .
Azure Communication Services	—	Email delivery for payslips and invites.
Jest + Supertest	—	Test tooling (configured; suite not yet written).

Frontend

Technology	Version	Why it is used
Next.js	16 (App Router)	React framework. Server Components, Server Actions, file-based routing.
React	19	UI library. Uses <code>useActionState</code> , <code>useOptimistic</code> , <code>useTransition</code> .
Tailwind CSS	4	Utility-first styling.
Radix UI	—	Unstyled, accessible component primitives (dialog, dropdown, tabs, etc.).
Animate UI	—	Animated component layer built on Radix, vendored into <code>components/animate-ui/</code> .
Motion (Framer Motion)	13	Animation engine behind Animate UI.
lucide-react	—	Icon set.
date-fns	4	Date formatting.
react-day-picker	10	Calendar / date picker.
shiki	4	Syntax highlighting in the styleguide.

Infrastructure

Technology	Purpose
Docker + Docker Compose	Runs PostgreSQL and the API. Postgres is published on 5433 , not 5432, so it cannot collide with another local Postgres.
npm workspaces	Monorepo — <code>api</code> , <code>frontend</code> , <code>shared</code> resolve to each other by symlink.

Q: Why NestJS and not plain Express?

Structure. NestJS gives modules, dependency injection, and a guard/pipe/filter pipeline. Authentication and permissions are registered **globally once** in `app.module.ts`, which means a newly added endpoint is **secured by default** — you have to explicitly mark it `@Public()` to open it. With plain Express you would have to remember to add middleware to every route.

Q: Why Prisma and not raw SQL or TypeORM?

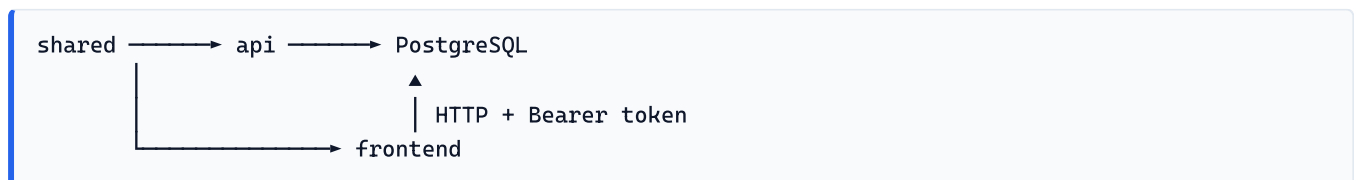
The schema is a single source of truth that generates both the migrations and a fully typed client. If a column is renamed, every query touching it becomes a compile-time error rather than a runtime one.

Q: Why Next.js App Router with Server Components?

The JWT never needs to reach the browser. Pages render on the Next.js server, which reads the `httpOnly` cookie and calls the API with it. There is no client-side data fetching layer, no token in `localStorage`, and no API keys in the bundle.

3. Architecture

Three packages, one repository



Package	npm name	Responsibility
api/	@peoplepay360/api	Every database read and write. Authorisation. Payroll computation.
frontend/	@peoplepay360/web	Rendering, forms, navigation. Holds a session cookie and an HTTP client — nothing more.
shared/	@peoplepay360/shared	Types, enums, RBAC matrix, pure domain maths. No Nest, Prisma, React or DOM dependency.

The API is the only thing that touches the database. The web client never opens a database connection. This means authorisation cannot be sidestepped by calling a different client — there is exactly one place to get it right.

Request flow, end to end

```
Browser
|
| requests a page, carries the pp360_token httpOnly cookie
|
▼
Next.js server (Server Component)
|
| reads the cookie, calls the API with Authorization: Bearer <jwt>
|
▼
NestJS
|
| 1. JwtAuthGuard      - verifies token, re-reads the user from the DB
| 2. PermissionsGuard - checks the shared RBAC matrix
| 3. ValidationPipe   - validates and strips the request body against the DTO
| 4. Controller → Service → Prisma
|
▼
PostgreSQL
|
| response bubbles back, Next.js renders HTML on the server and streams it
|
Browser
```

Q: Why does `shared` exist?

So the API and the web client cannot drift apart. Three concrete cases:

1. **DTO types** — if the API changes a response shape, the frontend gets a compile error rather than `undefined` at runtime.
2. **The RBAC matrix** — one table. The API consults it to *enforce*; the client consults it to decide what to *offer* (which sidebar links to show). One definition, so the menu can never disagree with what the API allows.
3. **Domain maths** — `computeWeeklyHours()` is used by the client to show a live preview as a schedule is edited, and by the API as the authority when saving. Same function, so preview and stored value cannot differ.

4. The database

PostgreSQL 16, accessed through Prisma 6

20 application tables and **15 enums**. Schema: `api/prisma/schema.prisma`.

The 20 tables

Access control (1)

#	Table	What it holds
1	User	Sign-in account: email, <code>passwordHash</code> , <code>role</code> , <code>active</code> , <code>mustChangePassword</code> , <code>invitedAt</code> .

HR master data (3)

#	Table	What it holds
2	Department	Engineering, Sales, etc.
3	JobPosition	Job titles.
4	Employee	The hub. Identity, contact, bank details, type, status, hire/exit dates. Foreign keys to department, position, manager (self-relation), working schedule, and a required 1-to-1 to User .

Working schedules (2)

#	Table	What it holds
5	WorkingSchedule	Named pattern — "Standard 9-6", with hoursPerWeek derived from its lines.
6	WorkingScheduleLine	One row per working day: dayOfWeek (0=Sun), startTime , endTime , breakHours .

Contracts (1)

#	Table	What it holds
7	Contract	dateStart / dateEnd (null = open-ended), status , wage , contract type, and optional overrides for job position, schedule and salary structure.

Attendance (1)

#	Table	What it holds
8	Attendance	checkIn , checkOut , derived workedHours / overtimeHours , derived status , plus a correction audit trail (manuallyEdited , editedById , editedAt , editReason).

Time off (3)

#	Table	What it holds
9	TimeOffType	Annual, Sick, Unpaid... with unit (DAY/HOUR), requiresAllocation , requiresApproval , paid , colorHex , maxDaysPerRequest .
10	LeaveAllocation	Granted quantity valid over a date range, with its own approval status.
11	LeaveRequest	The actual request: dates, derived duration , status, approval/refusal metadata, and an optional link to the allocation it consumed.

Salary configuration (2)

#	Table	What it holds
12	SalaryStructure	A named set of rules — "Regular Employee", "Intern".
13	SalaryRule	The heart of the payroll engine. code , category , sequence , computeType , and either a fixed amount, a percentage + base, or a formula. Plus an optional condition .

Payroll (3)

#	Table	What it holds
14	Payrun	A batch for a period: structure, <code>periodStart</code> / <code>periodEnd</code> , status, and the compute/validate/paid timestamps.
15	Payslip	One employee's result: worked days/hours, leave days, overtime, <code>basicWage</code> , <code>grossPay</code> , <code>totalDeductions</code> , <code>netPay</code> , and a <code>warnings</code> string array.
16	PayslipLine	One line per rule that appears on the payslip. <code>code</code> , <code>name</code> , <code>category</code> are denormalised so the payslip renders identically even if the rule is later edited.

Supporting (4)

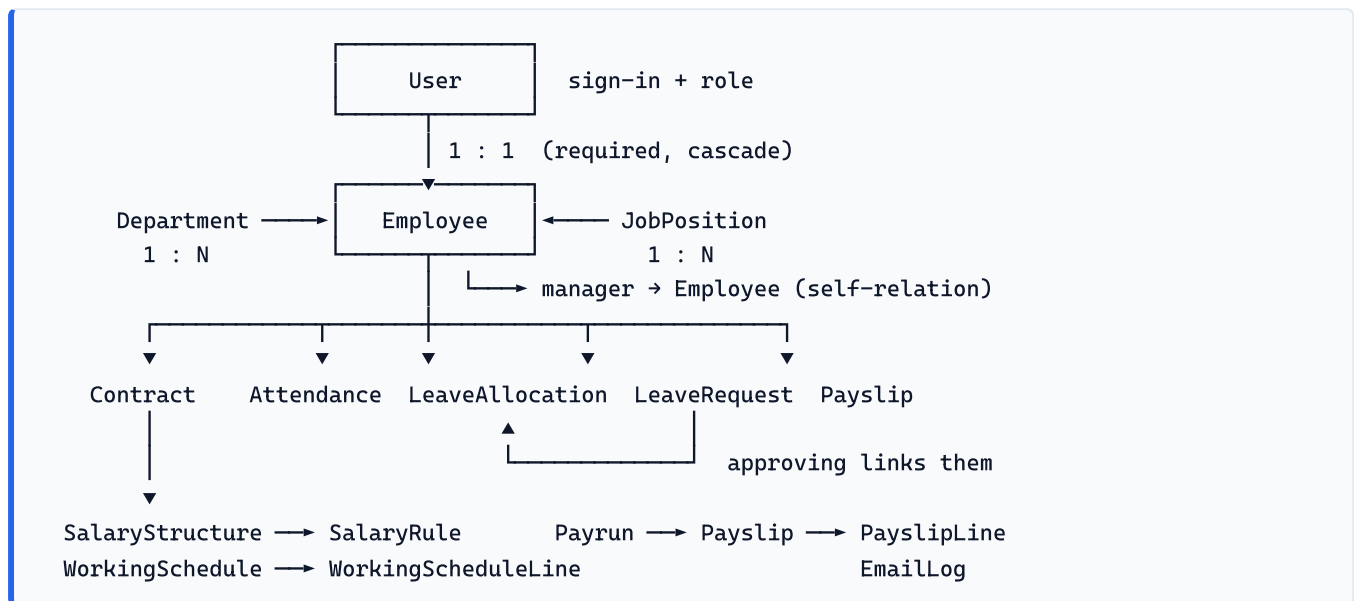
#	Table	What it holds
17	EmailLog	Every delivery attempt: recipient, subject, body, attachment name, status, error. The in-app outbox.
18	AuditLog	Who did what: user (nullable + denormalised name/role), action, entity, entity id and label, a JSON field-level <code>changes</code> diff, a <code>snapshot</code> for deletes, HTTP method, path, IP.
19	Notification	Per-recipient row: type, title, body, href, actor name, <code>readAt</code> .
20	AppSettings	A single pinned row (<code>id = "singleton"</code>) for org-wide settings: <code>maxCheckInsPerDay</code> , <code>warnOnCheckOut</code> .

The 15 enums

`EmployeeType`, `EmployeeStatus`, `ScheduleType`, `ContractStatus`, `ContractType`, `AttendanceStatus`, `LeaveUnit`, `AllocationStatus`, `LeaveRequestStatus`, `RuleCategory`, `ComputeType`, `PayrunStatus`, `PayslipStatus`, `EmailStatus`, `AuditAction`.

They are **native PostgreSQL enums** mirroring the const unions in `shared/src/enums.ts` one for one. The database rejects an invalid status outright.

Relationship summary



Key design decisions

Concern	Choice	Why
Primary keys	<code>uuid(7)</code> (UUIDv7)	Globally unique but time-sortable , unlike UUIDv4. Index locality is good, and ids do not leak a row count the way an auto-increment integer does.
Money	<code>Decimal(12,2)</code> → <code>NUMERIC</code>	A float cannot represent a rupee exactly. $0.1 + 0.2 \neq 0.3$. Payroll totals must reconcile to the cent.
Hours	<code>Decimal(8,2)</code>	Same reason — half-days and breaks are fractional.
Statuses	Native PG enums	The database itself rejects an invalid value.
Warnings	<code>String[]</code> (native <code>text[]</code>)	Read straight back as <code>string[]</code> with no JSON parse.
Audit diffs	<code>Json</code>	Field-level <code>{ field: { from, to } }</code> is a variable shape.

Q: Why is `Employee → User` required and 1-to-1?

So one person is one record **and** one account, never one without the other. Creating an employee creates the account in the same **database transaction**. An admin is an employee like everyone else. There is nothing to link manually, so nothing can drift out of sync, and every code path can assume the account exists.

Q: Why is `AuditLog.userId` nullable?

`onDelete: SetNull`. The trail has to **survive** the account it records. Deleting a user must not take their history with them — but it must also not leave rows that can no longer say who acted. That is why `userName` and `userRole` are **denormalised** onto the row: the id goes null, the names remain.

Q: Why does `PayslipLine` duplicate the rule's code and name?

So a payslip is **reproducible**. If someone renames the "HRA" rule to "House Rent Allowance" next year, payslips issued this year still say "HRA". A historical financial document must not change retroactively because configuration changed. The same reasoning puts `contractId` on `Payslip` as a snapshot.

5. Authentication and authorisation

Authentication — JWT in an `httpOnly` cookie

1. User posts email + password to `POST /api/auth/login`.
2. API looks up the user, compares with `bcrypt.compare`.
3. On success it signs a JWT containing `{ sub, email, name, role, employeeId }`.
4. `Next.js` stores it in a cookie named `pp360_token` with `httpOnly: true`, `sameSite: 'lax'`, `secure` in production, 7-day max age.
5. A second cookie `pp360_user` caches name and role so navigation renders without an extra round trip.

The token is never readable from client-side JavaScript. That is the point of `httpOnly` — it removes the entire class of XSS token theft. The `Next.js` server reads the cookie and attaches it as a bearer header when calling the API.

The same-message rule

Login returns the **same** "Invalid email or password." for an unknown email, a wrong password, *and* a deactivated account — so the endpoint cannot be used to enumerate which addresses are registered.

Q: Why re-read the user from the database on every request?

`jwt.strategy.ts` does a `findUnique` in `validate()`. A JWT is stateless — once signed it is valid until it expires. If an account is deactivated or its role is downgraded, a stale token would keep working for up to 7 days. Re-checking makes a change take effect **immediately**. The cost is one indexed primary-key lookup.

Authorisation — the RBAC matrix

5 roles × 13 modules × 5 actions, defined once in `shared/src/rbac.ts`.

Role	What it can reach
<code>EMPLOYEE</code>	Only their own rows. Read employees/contracts/schedules; create their own attendance and leave requests; read their own payslips.
<code>HR_MANAGER</code>	Full HR — employees, contracts, schedules, attendance, time off. No payroll at all.
<code>HR_PAYROLL_USER</code>	All HR, plus create/read/update on pay runs and payslips, read-only salary config, dashboard.
<code>HR_PAYROLL_MANAGER</code>	The above plus delete and approve on payroll and full salary configuration.

Role	What it can reach
ADMIN	Everything, plus read-only access to the audit trail.

The modules are: `employees`, `contracts`, `workingSchedules`, `attendance`, `timeOffRequests`, `timeOffAllocations`, `timeOffTypes`, `payruns`, `payslips`, `salaryStructures`, `salaryRules`, `dashboard`, `auditLogs`. The actions are: `read`, `create`, `update`, `delete`, `approve`.

Q: Why is HR Manager written out per-module rather than as a seniority ladder?

Because HR Manager has **full HR access but zero payroll access**. It is not "below" the payroll roles — it is beside them. A simple numeric level cannot express that, which is exactly why the matrix is a table.

Q: Is the audit log writable by an admin?

No — `READ` only, for everyone including the admin. *A trail that can be edited or pruned by the people it records is not a trail.*

Two enforcement layers

```
Client → can(role, module, action) → hides the sidebar link and the button
API    → PermissionsGuard          → 403 Forbidden
```

The client check decides what is **offered**; the API check is the **boundary**. A forged request with a hand-crafted URL still fails at the guard.

Record scoping

The matrix says *which modules*; it does not say *which rows*. An `EMPLOYEE` reading `/employees` is additionally narrowed by the service layer to rows carrying their own `employeeId` — `scopeToOwnRecords(role)` plus a `NO_MATCH_ID` sentinel in `common/scoping.ts` for the case where there is no employee link at all.

Other security measures

- **Global** `ValidationPipe` with `whitelist: true` and `forbidNonWhitelisted: true` — an unknown property in a request body is **rejected outright**, not silently stripped. A client cannot smuggle `role: 'ADMIN'` past a DTO.
- **Helmet** for security headers, with `crossOriginResourcePolicy` relaxed so payslip PDFs can be embedded cross-origin.
- **CORS** restricted to configured origins.
- **The API refuses to boot in production** if `JWT_SECRET` is under 32 characters — a weak secret would let anyone mint a valid admin token.
- **Bcrypt** with 10 salt rounds; passwords are never stored or logged in plain text.

- `mustChangePassword` — a system-issued password works once; the app shell will not render until it is replaced.
- **Formula sandbox** — see §6.

6. The payroll engine — the technical centrepiece

This is the part most worth being able to explain. **Salary rules are data, not code.**

The problem it solves

A hardcoded payroll (`net = basic + hra - pf - tax`) means a code change and a redeploy every time a company adds an allowance or a tax slab changes. Here, a salary rule is a **row in a table** that an authorised payroll manager edits through the UI.

A `SalaryRule` row

Column	Purpose
<code>code</code>	Identifier referenced inside other formulas — <code>BASIC</code> , <code>HRA</code> , <code>GROSS</code> .
<code>category</code>	<code>BASIC</code> / <code>ALLOWANCE</code> / <code>GROSS</code> / <code>DEDUCTION</code> / <code>CONTRIBUTION</code> / <code>NET</code> . Drives totals and sign.
<code>sequence</code>	Execution order. Lower runs first.
<code>computeType</code>	<code>FIXED</code> , <code>PERCENTAGE</code> , or <code>FORMULA</code> .
<code>amountFixed</code>	For <code>FIXED</code> .
<code>amountPercentage</code> + <code>percentageBase</code>	For <code>PERCENTAGE</code> — e.g. 40% of <code>BASIC</code> .
<code>formula</code>	For <code>FORMULA</code> — a JS expression.
<code>condition</code>	Optional guard; the rule is skipped when it evaluates false.
<code>appearsOnPayslip</code>	Whether it produces a visible line, or is only an intermediate.

How it runs — payroll-engine.service.ts

1. Filter to active rules, sort by sequence (then code, for a stable tie-break).
2. Build a scope object from the payroll context:
wage, workedDays, workedHours, leaveDays, paidLeaveDays,
unpaidLeaveDays, overtimeHours, scheduledDays, scheduledHours, employeeType
3. For each rule, in order:
 - a. If it has a condition, evaluate it. False → set scope[code] = 0, skip.
 - b. Compute the amount by computeType.
 - c. Round to 2 decimals.
 - d. *****Write the result back into the scope under the rule's code.*****
 - e. If appearsOnPayslip, emit a line.
4. Sum by category → basicWage, grossPay, totalDeductions, netPay.

Step 3d is the whole idea. Because each result enters scope under its own code before the next rule runs, a later rule can reference an earlier one. **NET** is literally the string **GROSS - PF - PT - TDS - ULD** stored in a database column — not a hardcoded subtraction.

The seeded "Regular Employee" structure — a worked example

Seq	Code	Category	Type	Definition
10	BASIC	BASIC	FORMULA	<code>scheduledDays > 0 ? wage * Math.min(1, (workedDays + paidLeaveDays) / scheduledDays) : wage</code>
20	HRA	ALLOWANCE	PERCENTAGE	40% of BASIC
30	TA	ALLOWANCE	FIXED	2400
40	MA	ALLOWANCE	FIXED	1800
50	OT	ALLOWANCE	FORMULA	<code>overtimeHours * (BASIC / 160) * 1.5</code>
100	GROSS	GROSS	FORMULA	<code>BASIC + HRA + TA + MA + OT</code>
110	PF	DEDUCTION	FORMULA	<code>Math.min(BASIC, 15000) * 0.12</code>
120	PT	DEDUCTION	FIXED	200
130	TDS	DEDUCTION	FORMULA	<code>GROSS * 12 > 1000000 ? GROSS * 0.15 : (GROSS * 12 > 500000 ? GROSS * 0.08 : 0)</code>
140	ULD	DEDUCTION	FORMULA	<code>scheduledDays > 0 ? (wage / scheduledDays) * unpaidLeaveDays : 0</code>
200	NET	NET	FORMULA	<code>GROSS - PF - PT - TDS - ULD</code>

Read it top to bottom and every real payroll concept is visible: **pro-rata basic** by attendance, a **percentage allowance**, **overtime at 1.5×**, a **PF ceiling** of 15,000, **progressive TDS slabs**, and an **unpaid-leave deduction** — all of it configuration, none of it code.

The "Intern" structure is the same engine with 4 rules: pro-rata **BASIC**, **GROSS = BASIC**, a flat **PT** of 200, **NET = GROSS - PT**.

Q: You use `new Function()` to evaluate formulas. Isn't that dangerous?

It is the one genuinely sharp edge, and it is defended in three layers:

1. **Denylist** — the expression is regex-tested for `require`, `import`, `process`, `globalThis`, `eval`, `Function`, `constructor`, `__proto__`, `prototype`, `fetch`, `window`, `document`, `module`, `exports`, `child_process`, `global`. A match throws before evaluation.
2. **Shadowing** — those globals are passed in as **parameters bound to** `undefined`, so even a name that slips the regex resolves to nothing rather than to the host object.
3. **Scope restriction** — only the payroll context variables and `Math` are in scope. It runs under `"use strict"`.

Plus: formulas can only be written by an authenticated `HR_PAYROLL_MANAGER`, and `validateExpression()` runs the formula against **probe values** before it is stored — so a syntactically broken rule is rejected at save time rather than breaking a whole pay run later.

The honest answer if pushed: a fully isolated VM (`node:vm` with a timeout, or a dedicated expression parser) would be stronger, because this does not defend against an infinite loop. It is a known, bounded trade-off — the input is privileged, and the alternative is hardcoding the rules.

Q: Why can deductions be stored as positive numbers?

The **magnitude** goes into scope under the rule's code; the **sign** comes from the category. So a formula can read `GROSS - PF` naturally instead of `GROSS + PF` with a negative PF, which would be confusing to write.

Q: What if a structure has no explicit GROSS or NET rule?

The engine falls back to deriving them — `gross = basic + allowances`, `net = gross - deductions`. Every structure yields a coherent payslip either way.

7. Core functionality, module by module

7.1 Employee management

- Full CRUD, paginated and filterable list, plus a detail page with tabs for contracts, attendance, time off and payslips.
- **Creating an employee creates the sign-in account in the same transaction**, generates a one-time password and emails an invite.
- Auto-generated sequential `employeeCode`.
- Self-relation manager hierarchy.
- **Deactivate rather than delete** once payroll history exists — payslips do not cascade, so deleting would orphan financial records.
- An admin cannot invite or delete **themselves**.

- Re-invite endpoint reissues a one-time password.

7.2 Contracts

- Period-scoped: an employee may hold many over time.
- **Overlapping `RUNNING` contracts are rejected at save time** — otherwise payroll could not tell which wage applies.
- `resolveContractForPeriod()` picks the one `RUNNING` contract overlapping the payroll period; if legacy data somehow yields several, the **latest start wins** as the most recently agreed terms.
- A contract can override the employee's job position, working schedule and salary structure.

7.3 Working schedules

- A pattern of lines: day of week, start, end, break hours.
- `hoursPerWeek` **is always derived**, never accepted from a client.
- **Overnight shifts roll over correctly** — `lineHours()` treats an end time at or before the start as crossing midnight: `24 - start + end`.

7.4 Attendance

- Check-in / check-out punches, one row per punch.
- **Everything is derived on write** by `computeAttendance()` :
 - `workedHours` = raw span – break (only deducted if the shift ran longer than the break itself).
 - `overtimeHours` = worked – expected for that weekday.
 - `status` : `LATE` if the check-in exceeds the scheduled start plus a grace period; `HALF_DAY` if worked is under half the expected hours; `MISSING_CHECKOUT` for an open punch **only once the day has passed** — an open check-in during the same day is simply `PRESENT`, not an error.
- **Manual corrections are audited** — `manuallyEdited`, `editedById`, `editedAt` and a required `editReason`.
- **Self-service punch card** with a configurable daily cap (`maxCheckInsPerDay`), and an optional confirmation before closing a shift.

7.5 Time off

- Types are configurable: unit, whether an allocation is required, whether approval is required, whether it is **paid** (which is what payroll reads).
- **Balance is derived, never stored**: approved allocations valid on the date, minus approved requests. Pending requests are reported separately so an employee sees what is still awaiting a decision.
- **Duration counts only scheduled working days** — a Friday-to-Monday request on a Mon–Fri schedule costs **two** days, not four.
- Validation on submit: overlap with an existing request, `maxDaysPerRequest`, and sufficient balance. The error distinguishes "never allocated" from "allocated but not valid on these dates", which is otherwise baffling for a future-dated request.

- **Approving links the request to a specific allocation** (the valid one soonest to expire), so consumption is auditable rather than inferred later.
- A type with history is **archived**, not deleted, so existing requests keep their label.

7.6 Payroll — the pay run lifecycle



Step	What happens
Eligibility	Before creating a run, <code>GET /payruns/eligible-employees</code> reports who qualifies and why not — no running contract, or an existing payslip for an overlapping period. It also surfaces non-blocking warnings such as missing bank details or a contract that normally uses a different structure.
Create	A Payrun plus one DRAFT payslip per explicitly selected employee. Amounts are all zero at this point.
Compute	For each payslip: resolve the contract, pull real attendance and approved leave for the period, count scheduled days from the schedule, then run the structure's rules in sequence.
Validate	Blocked by <code>detectWarnings()</code> if any blocking condition is present.
Mark paid	Terminal. A PAID run can no longer be recomputed.
Send payslips	Generates a PDF per payslip and emails it; every attempt is logged.

The four blocking warnings — these stop validation until resolved:

1. **Duplicate payslip** — the employee already has one for an overlapping period in another run.
2. **Missing bank details** — payment cannot be released.
3. **No applicable contract** for the period.
4. **Negative net pay** — deductions exceed gross; something is wrong.

Non-blocking warnings are recorded on the payslip but do not stop the run — no attendance recorded, unpaid leave reducing pay, no working schedule assigned (scheduled days then estimated at 22).

Q: Why are payslips created as DRAFT with zero amounts, then computed?

It separates **who is in the run** from **what they are paid**. The selection is deliberate and reviewable, and compute can be re-run any number of times as attendance or leave is corrected — right up until the run is validated.

7.7 PDF generation

`pdf.service.ts` builds an A4 payslip with **PDFKit** — branded header, employee and period details, the earnings and deductions tables from `PayslipLine`, and the net pay total. Generated on demand and streamed.

The PDF proxy problem, and its solution: a browser cannot set an `Authorization` header on a plain `<a href>` link, and the JWT is `httpOnly` so JavaScript cannot read it to fetch the file either. So `/api/payslips/[id]/pdf` in `Next.js` is a small proxy route: it reads the cookie server-side, calls the NestJS endpoint with a bearer header, and streams the body back. The API still enforces who may read that payslip.

7.8 Email delivery

Provider is chosen by **what is configured**, not by a flag:

Condition	Transport
<code>ACS_EMAIL_CONNECTION_STRING</code> is set	Azure Communication Services
Only <code>SMTP_HOST</code> is set	SMTP
Neither	Outbox — nothing leaves; the attempt is recorded

The outbox mode is deliberate: the whole flow is demonstrable **without credentials**. Every attempt lands in `EmailLog` with its status and, on failure, the provider's own error message.

7.9 Audit trail

Implemented as a **Prisma query extension** (`audit.extension.ts`), not as manual logging in each service.

Q: Why a Prisma extension rather than logging in each service?

A query extension runs on **either side of every database operation**, which makes it the one place that can read a row *before* a write lands and compare it with what the write returned — producing a real field-level diff. Asking each service to log its own changes would leave the trail *exactly as complete as everyone's memory*, which is to say not.

It records: action, entity, entity id and a human-readable label, a `changes` diff for updates, a full `snapshot` for deletes (nothing else could recover the row), and the HTTP method, path and IP from an `AsyncLocalStorage` request context.

7.10 Notifications

- Per-recipient rows, so read state is simply a `readAt` column rather than a join table.
- **Live delivery over Server-Sent Events** (`GET /notifications/stream`).
- Since `EventSource` cannot set an `Authorization` header either, the web client proxies the stream through its own server and attaches the cookie there.
- The stream implementation handles the awkward parts: a heartbeat comment line every interval so an idle proxy does not close the connection, `X-Accel-Buffering: no` and `no-transform` so nothing buffers it, and teardown on both `close` and `error` so a dropped connection cannot leak a listener and a timer — or take the process down with an unhandled write error.

7.11 Dashboard

Aggregate counts and trends — headcount, attendance summary, pending leave requests, recent pay runs. Read-only, and only for roles with `dashboard` access.

8. Folder and file reference

Root

Path	Purpose
<code>package.json</code>	npm workspace root. All the orchestration scripts.
<code>docker-compose.yml</code>	Postgres + API services.
<code>README.md</code>	Getting started, credentials, commands.
<code>docs/</code>	This guide and the five deeper documents.
<code>api/</code> , <code>frontend/</code> , <code>shared/</code>	The three workspaces.
<code>apps/</code> , <code>prisma/dev.db</code>	Stale artefacts from an earlier layout — not used by the running app. Safe to ignore or delete.

`shared/src/`

File	Contents
<code>index.ts</code>	Re-exports everything.
<code>enums.ts</code>	Const-object enums plus their display labels. Plain objects rather than TS <code>enum</code> s so values survive JSON transport and can be iterated to build a <code><select></code> .
<code>types.ts</code>	Every DTO crossing the wire. Dates are ISO strings — neither side pretends they are <code>Date</code> objects.
<code>rbac.ts</code>	The permission matrix, <code>can()</code> , <code>scopeToOwnRecords()</code> , <code>visibleModules()</code> .
<code>domain.ts</code>	Pure maths — <code>computeWeeklyHours</code> , <code>lineHours</code> , <code>workingDaysInRange</code> , <code>computeLeaveDuration</code> , <code>resolveContractForPeriod</code> , <code>rangesOverlap</code> , <code>round2</code> . No I/O.

`api/`

Path	Purpose
<code>prisma/schema.prisma</code>	The data model — 20 models, 15 enums.
<code>prisma/migrations/</code>	Versioned SQL migrations.
<code>prisma/seed.ts</code>	Seed script.

Path	Purpose
<code>src/main.ts</code>	Bootstrap: global <code>/api</code> prefix, Helmet, CORS, <code>ValidationPipe</code> , Swagger at <code>/api/docs</code> .
<code>src/app.module.ts</code>	Root module. Registers <code>JwtAuthGuard</code> and <code>PermissionsGuard</code> as global guards and the exception filter.
<code>src/health.controller.ts</code>	<code>/api/health</code> — what the Docker healthcheck polls.
<code>src/config/configuration.ts</code>	Typed config from environment, including the production <code>JWT_SECRET</code> length check.
<code>src/prisma/prisma.service.ts</code>	Prisma client as an injectable, with the audit extension applied.
<code>src/common/decimal.ts</code>	<code>toNumber</code> / <code>toDecimal</code> / <code>round2</code> — the Prisma <code>Decimal</code> ↔ JS <code>number</code> boundary.
<code>src/common/pagination.ts</code>	<code>pageArgs</code> / <code>paginated</code> — one pagination shape for every list.
<code>src/common/scoping.ts</code>	Record scoping for the Employee role, and the <code>NO_MATCH_ID</code> sentinel.
<code>src/common/decorators/</code>	<code>@Public</code> , <code>@CurrentUser</code> , <code>@RequirePermission</code> .
<code>src/common/filters/all-exceptions.filter.ts</code>	Uniform error responses.
<code>src/common/one-time-password.ts</code>	Invite password generation.
<code>src/common/validation/entity-id.ts</code>	Id parameter validation.

`api/src/modules/` — each has `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/`

Module	Key files and what they do
<code>auth/</code>	<code>auth.service.ts</code> (login, change password, me), <code>jwt.strategy.ts</code> (verify + re-read the user), <code>guards/jwt-auth.guard.ts</code> , <code>guards/permissions.guard.ts</code> .
<code>employees/</code>	CRUD, the transactional create-with-account, re-invite, options list.
<code>contracts/</code>	CRUD plus the overlap guard and <code>getContractForPeriod</code> .
<code>attendance/</code>	CRUD, <code>computeAttendance</code> , the punch card, period summaries, <code>getWorkedTimeInPeriod</code> (consumed by payroll).
<code>time-off/</code>	Types, allocations, requests, approve/refuse/cancel, <code>getBalances</code> , <code>approvedLeaveDaysInPeriod</code> (consumed by payroll). The largest service at ~870 lines.
<code>payroll/</code>	<code>payroll-engine.service.ts</code> (the rule evaluator), <code>payslips.service.ts</code> (<code>computeFor</code> gathers the context), <code>payruns.service.ts</code> (lifecycle + warnings), <code>salary-config.service.ts</code> (structures and rules), <code>pdf.service.ts</code> , <code>mail.service.ts</code> .
<code>config/</code>	Working schedules, departments, job positions, and the <code>AppSettings</code> singleton.
<code>dashboard/</code>	Aggregate statistics.
<code>notifications/</code>	List, mark read, and the SSE stream.

Module	Key files and what they do
audit/	<code>audit.extension.ts</code> (the Prisma hook), <code>audit.diff.ts</code> (field diffing), <code>audit.entities.ts</code> (which models are audited), <code>audit-context.ts</code> (<code>AsyncLocalStorage</code> request context), <code>audit.interceptor.ts</code> , and the read-only controller.

frontend/src/

app/ — routes (Next.js App Router)

Route group	Contents
(app)/	The admin/HR shell — sidebar, breadcrumbs, notification bell, theme toggle. Its <code>layout.tsx</code> is also the gate: no session → <code>/login</code> ; <code>mustChangePassword</code> → <code>/change-password</code> . Contains <code>page.tsx</code> (overview), <code>employees/</code> , <code>contracts/</code> , <code>attendance/</code> , <code>time-off/</code> , <code>payruns/</code> , <code>payslips/</code> , <code>salary/</code> , <code>settings/</code> , <code>audit/</code> , <code>profile/</code> .
(me)/me/	The self-service space for the Employee role — punch card, own attendance, own leave, own payslips, own profile. Separate because every list in the admin panel would hold exactly one line for them.
login/ , change-password/	Unauthenticated routes.
api/payslips/[id]/pdf/route.ts	The PDF proxy that attaches the cookie.
api/notifications/stream/route.ts	The SSE proxy that attaches the cookie.
styleguide/	A live component gallery.

The per-route file convention — the pattern repeated on every screen:

File	Role
page.tsx	A Server Component . Fetches data and renders.
fields.ts	The <code>FieldSpec[]</code> describing this resource's form fields.
actions.ts	Server Actions — the create/update/delete mutations.
_components/	Client components local to that route.
loading.tsx	Skeleton shown while the server component streams.
error.tsx , not-found.tsx	Error boundaries.

components/

Folder	Contents
ui/	The base library — button, input, select, card, badge, dialog, calendar, date-picker, toast, theme, etc.
animate-ui/	Vendored animated components, split into <code>primitives/</code> (behaviour, built on Radix + Motion), <code>components/</code> (styled), and <code>icons/</code> .

Folder	Contents
<code>data/</code>	List rendering — <code>data-table.tsx</code> , <code>filter-bar.tsx</code> , <code>pagination.tsx</code> , <code>status-badge.tsx</code> , <code>skeletons.tsx</code> .
<code>form/</code>	The write spine — <code>record-form.tsx</code> , <code>record-dialog.tsx</code> , <code>row-actions.tsx</code> , <code>action-button.tsx</code> , <code>warning-dialog.tsx</code> .
<code>app/</code>	The shell — <code>app-sidebar.tsx</code> (built from the RBAC matrix), <code>app-breadcrumbs.tsx</code> , <code>notification-bell.tsx</code> .
<code>auth/</code>	<code>login-form.tsx</code> , <code>auth-shell.tsx</code> .

lib/

File	Purpose
<code>api-client.ts</code>	The server-side HTTP client. Builds URLs, attaches the bearer token from the cookie, throws a typed <code>ApiError</code> .
<code>session.ts</code>	<code>login</code> , <code>logout</code> , <code>getSession</code> (cookie-cached), <code>verifySession</code> (asks the API), <code>refreshSession</code> .
<code>access.ts</code>	Route guards and <code>landingFor(user)</code> — each role lands on the first screen it can actually open.
<code>fields.ts</code>	The <code>FieldSpec</code> type and <code>readForm()</code> . One spec drives both sides: the client renders the control, the server action reads the value back out. Adding a column is one line, not an edit in three files.
<code>mutate.ts</code>	The mutation helper. Every action returns one <code>FormState</code> shape — <code>{ ok, error, fieldErrors, message, id, record, warning }</code> — so <code>useActionState</code> looks identical on every screen. Also maps a write to the cache tags it must invalidate.
<code>paged.ts</code> , <code>refs.ts</code> , <code>format.ts</code> , <code>status.ts</code> , <code>utils.ts</code>	Pagination helpers, cached reference lists, formatters, status→colour mapping, <code>cn()</code> .

Q: Why are there two route groups, `(app)` and `(me)` ?

Because the two audiences need genuinely different interfaces. An HR manager needs filterable tables across all employees. An individual employee needs a punch card and their own three lists — every table in the admin panel would show them exactly one row. `landingFor()` sends anyone scoped to their own records straight to `/me`.

Q: What is a Server Action and why use one instead of an API route?

A function marked `"use server"` that a form can post to directly. React sends the `FormData` to the server, runs the function there, and returns the result to `useActionState`. No `fetch` call, no JSON serialisation, no client-side handler — and the API token stays server-side throughout.

9. Notable technical decisions — the likely questions

Q: Why derive so much instead of storing it?

Derived values cannot go stale. Weekly hours, leave balance, worked hours and overtime are all computed from their inputs. If a stored balance and the allocations it came from ever disagreed, there would be no way to know which is right. Deriving removes the question.

The exception is `PayslipLine`, which **is** denormalised — because a historical financial document must *not* change when configuration does. The rule is: **derive live operational data, snapshot financial history**.

Q: How is money handled and why does it matter?

Every money column is `Decimal(12,2)` → PostgreSQL `NUMERIC`. Floating point cannot represent decimal fractions exactly, so summing hundreds of payslip lines as floats would drift. `common/decimal.ts` normalises at the API boundary and `round2()` is applied after every rule computation.

Q: What happens if two contracts overlap?

Save is rejected. `resolveContractForPeriod` also defends against legacy data by taking the latest start date, but the guard is the real answer.

Q: How does pagination work?

`common/pagination.ts` provides `pageArgs` (turning page/pageSize into Prisma `skip` / `take`, capped by `MAX_PAGE_SIZE`) and `paginated` (wrapping results with total and page metadata). Every list endpoint returns the same `Paginated<T>` shape, so `components/data/` renders any of them.

Q: How is caching handled on the frontend?

Page data is per-request and **uncached** (`cache: "no-store"`) — it sits behind a session and must be current. **Reference lists** (departments, positions, schedules) are cached with tags, and `mutate.ts` maps each write path to the tags it invalidates. Server Actions bypass the client router cache, so a write also calls `revalidatePath` on the route tree.

Q: What is not done yet?

Stated plainly, because it is better to name it than be caught by it:

- **No test suite.** Jest and Supertest are configured and `npm test` is wired up, but no `.spec.ts` files exist. The payroll engine is the obvious first target — it is pure, deterministic and the highest-risk code in the project.
- **No refresh-token rotation.** A 7-day JWT; sign-in again after that. The per-request database check is what mitigates a stolen or stale token.
- **The formula sandbox has no execution timeout** — see §6.

- **No scheduled jobs.** Contracts do not auto-expire and allocations do not auto-close; both are driven by status columns that currently change only through the UI.
- **The web container is commented out** in `docker-compose.yml`, deliberately — `npm run dev:web` on the host is a hot reload instead of a two-minute image rebuild per change. Uncommenting the block restores the full stack.
- Roadmap: approval chains with delegation, payroll journal export, document storage, multi-currency and multi-company, statutory reports.

10. Quick facts sheet

Question	Answer
Backend framework	NestJS 11 (Node.js, TypeScript)
Frontend framework	Next.js 16 , App Router, React 19
Database	PostgreSQL 16
ORM	Prisma 6
Number of tables	20 (+ Prisma's own <code>_prisma_migrations</code>)
Number of enums	15
Styling	Tailwind CSS 4
UI components	Radix UI primitives + Animate UI , animated with Motion
Authentication	JWT (7-day) in an httpOnly cookie , <code>bcrypt</code> password hashing
Authorisation	RBAC matrix — 5 roles × 13 modules × 5 actions, defined once in <code>shared/</code>
API style	REST , all routes under <code>/api</code> , documented by Swagger at <code>/api/docs</code>
PDF generation	PDFKit
Email	Azure Communication Services , SMTP fallback, in-app outbox otherwise
Live updates	Server-Sent Events for notifications
Repository layout	npm workspaces monorepo — <code>api</code> , <code>frontend</code> , <code>shared</code>
Containerisation	Docker Compose (Postgres on port 5433)
Ports	Web 3000 , API 4000 , Postgres 5433

Question	Answer
Seed data	28 employees — 25 workforce people across 5 departments plus 3 demo staff accounts (<code>admin@</code> , <code>payroll@</code> , <code>hr@</code>); the <code>employee@</code> account is one of the 25. Also 16 job positions, 4 working schedules, 47 contracts (including expired ones, so period resolution is observable), ~1,860 attendance records over three months, 4 leave types with 67 allocations and 64 requests, 2 salary structures with 15 rules, and 2 completed pay runs producing 52 payslips and 485 payslip lines.
Seed password	<code>password123</code> for every account

Some seed data is deliberately imperfect, so the warning paths are demonstrable rather than theoretical. If an evaluator asks why a record looks wrong, this is the answer: two employees have **no bank details** (which is what blocks payroll validation), several attendance records are **missing a check-out** or were **manually corrected**, some contracts **expire within 30 days**, and the schedules include a **night shift** so the overnight-rollover logic is exercised.

The four demo accounts

Email	Role	Reaches
<code>admin@peoplepay360.com</code>	Admin	Everything, plus the audit trail
<code>payroll@peoplepay360.com</code>	HR Payroll Manager	All HR, plus payroll and salary configuration
<code>hr@peoplepay360.com</code>	HR Manager	All HR. No payroll at all
<code>employee@peoplepay360.com</code>	Employee	Only their own records — lands on <code>/me</code>

If you are asked for one thing to demonstrate

Open **Salary** → **the Regular Employee structure**, show that the 11 rules are editable rows with sequences and formulas, then run a pay run and open the resulting payslip. The lines on the payslip correspond one-to-one to those rows, computed against real attendance. That single walk-through demonstrates the data-driven engine, the derived-not-entered principle, and the whole pay run lifecycle at once.

11. Five-minute walkthrough script

1. **Sign in as the Employee** → lands on `/me` , not the admin panel. Punch in on the card, file a leave request. Point out the balance is derived.
2. **Sign in as HR Manager** → the sidebar has **no Payroll section at all**. That is the RBAC matrix rendering itself. Approve the leave request.
3. **Sign in as Payroll Manager** → Salary → open the Regular Employee structure. Walk the 11 rules top to bottom: pro-rata basic, 40% HRA, overtime at 1.5×, PF ceiling, TDS slabs, and `NET = GROSS - PF - PT - TDS - ULD` as a stored string. Emphasise: **none of this is in the code**.

4. **Create a pay run** → show the eligibility screen explaining who cannot be included and why. Compute.
Open a payslip and match its lines to the rules. Show a blocking warning (missing bank details) preventing validation.
5. **Download the PDF**, then **send the payslips** and show the outbox in Email logs.
6. **Sign in as Admin** → the Audit trail, showing the field-level diff of a change made earlier in the demo.